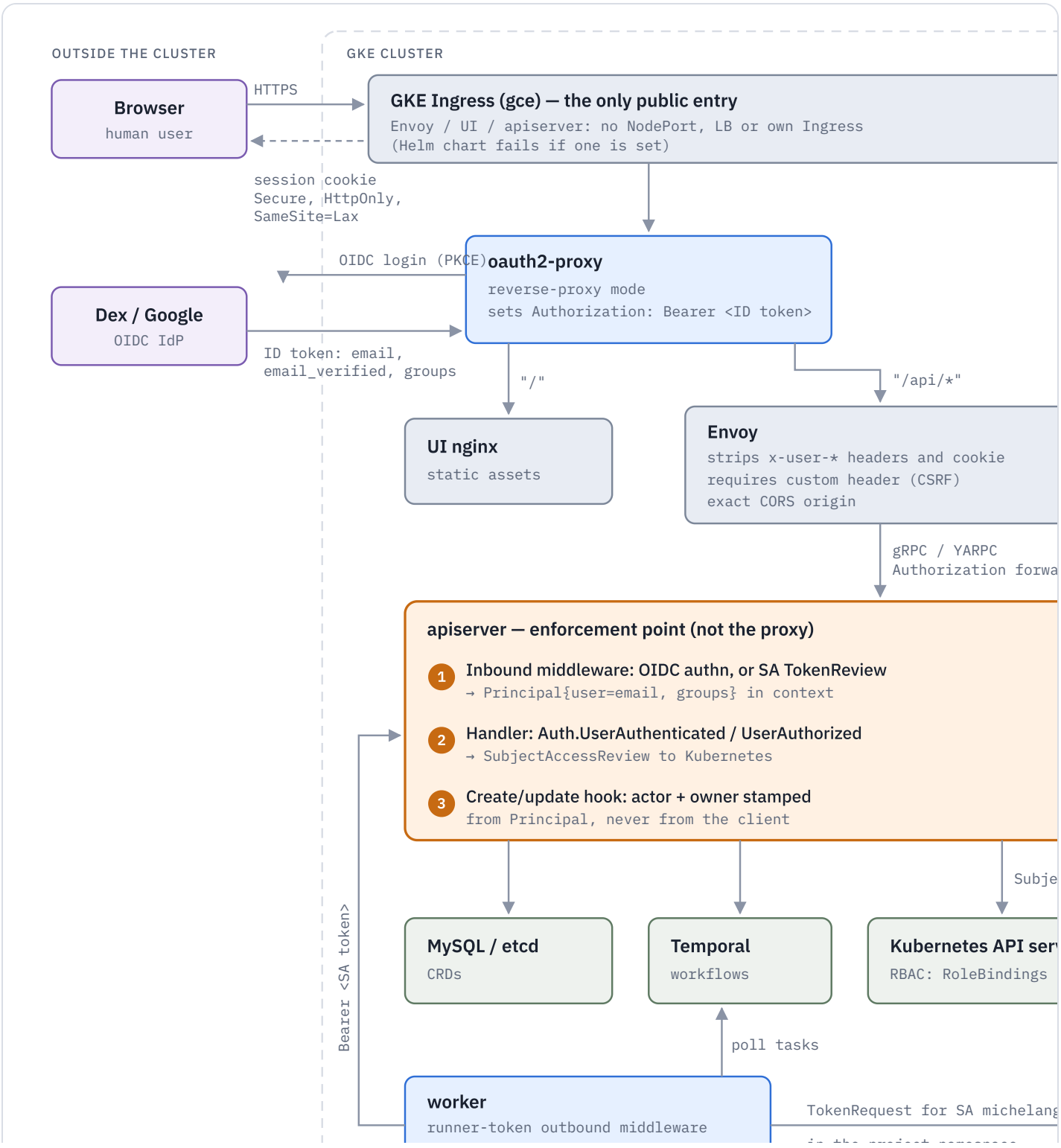
Michelangelo Auth Architecture

Request path from the browser to the apiserver on GKE, and how workers call back in with a service-account token. oauth2-proxy authenticates; the apiserver is where access is enforced.

Edge / routing Authentication Enforcement point Backends External



short-lived SA token per call

in the project namespace

Security properties

One public door

Only the GKE Ingress is exposed. Envoy, UI and apiserver get no NodePort, load balancer or Ingress of their own.

Authenticate at the edge, authorize at the apiserver

oauth2-proxy only establishes identity. The apiserver validates the token and runs SubjectAccessReview against Kubernetes RBAC.

No spoofed identity

Envoy strips `x-user-*` headers and the cookie, and requires a custom header plus an exact CORS origin to block CSRF.

NetworkPolicy

Only oauth2-proxy can reach Envoy and the UI. Only Envoy and the worker can reach the apiserver.

Workers use the same path

The worker requests a token for the `michelangelo-runner` service account and calls the apiserver with it, so it is checked by TokenReview and RBAC like any user.

Trusted ownership

Actor and owner on created or updated resources come from the authenticated Principal, never from request fields.